SYSTEM DESIGN

Horizontal vs Vertical Scaling Visual Guide

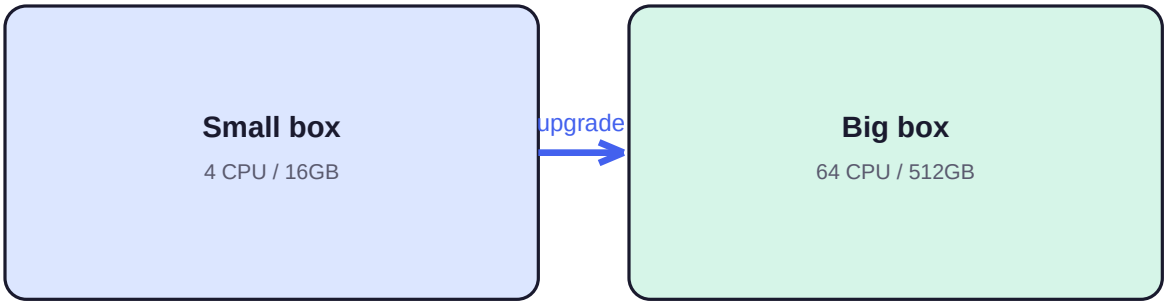
Bigger box or more boxes:
stateless design, load balancers, and sharding.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Scale Up (Vertical)

Bigger machine: more CPU, RAM, disk. Simple -- until physics says no.

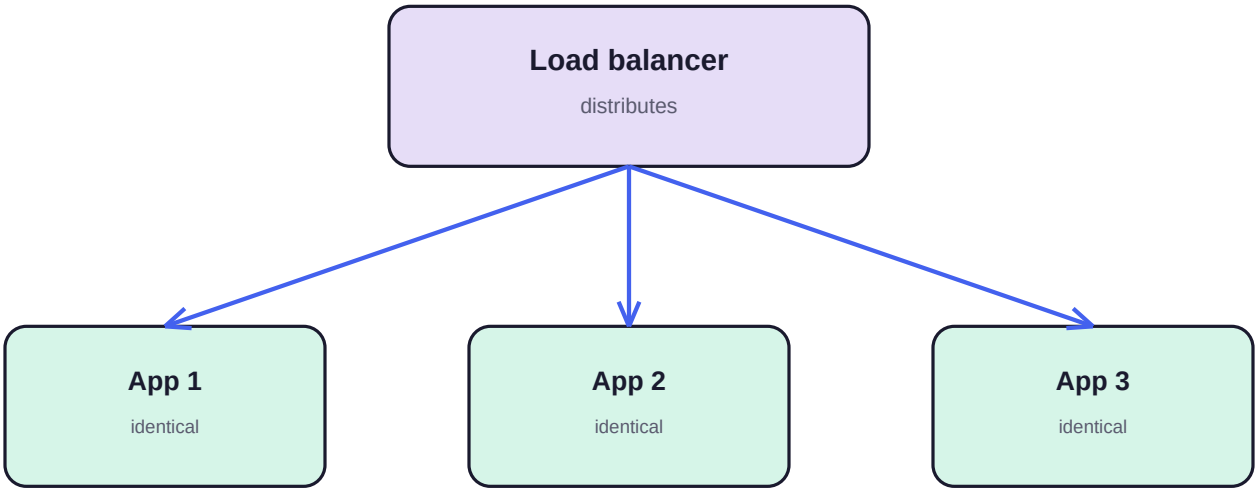


- Pros: zero architecture change, no distributed-systems pain, great for databases (for a while).
- Cons: hard ceiling (biggest SKU), downtime to resize, cost grows superlinearly, single point of failure.
- Always scale up FIRST: it is cheaper than re-architecting until you truly outgrow one box.

STEP 2/6

Scale Out (Horizontal)

More machines behind a load balancer: capacity grows with count.

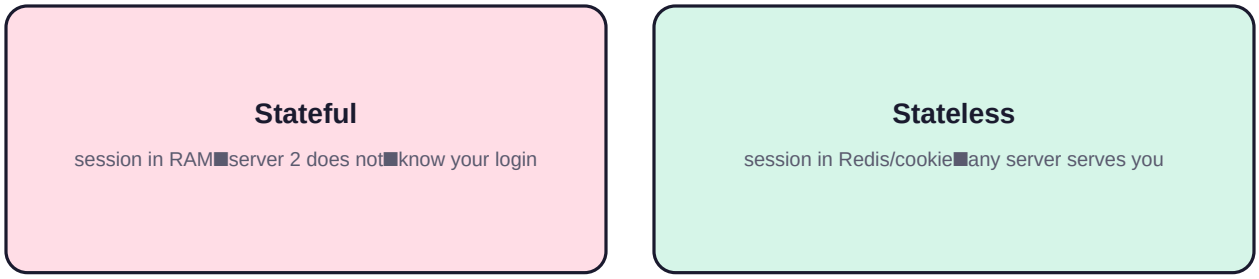


- Pros: near-linear capacity, no single point of failure, commodity hardware, elastic (add/remove).
- Cons: your app must be STATELESS, sessions externalized, deploys coordinated.
- The load balancer is now critical infra: run it redundant, health-check backends.

STEP 3/6

Stateless: The Hard Requirement

Any request can hit any server -- so servers may remember nothing.



- Externalize: sessions -> Redis, uploads -> object storage, jobs -> queue.
- Sticky sessions are a crutch: they break even balancing and complicate failover -- avoid.
- 12-factor rule: processes are disposable; all state lives in backing services.

STEP 4/6

The Load Balancer's Job

More than a traffic cop: health, routing smarts, and TLS.

Algorithms

round-robin, least-connections, consistent-hash (for caches)

Health checks

pull sick backends out of rotation automatically

TLS termination

decrypt once at the edge; plain HTTP inside (or re-encrypt)

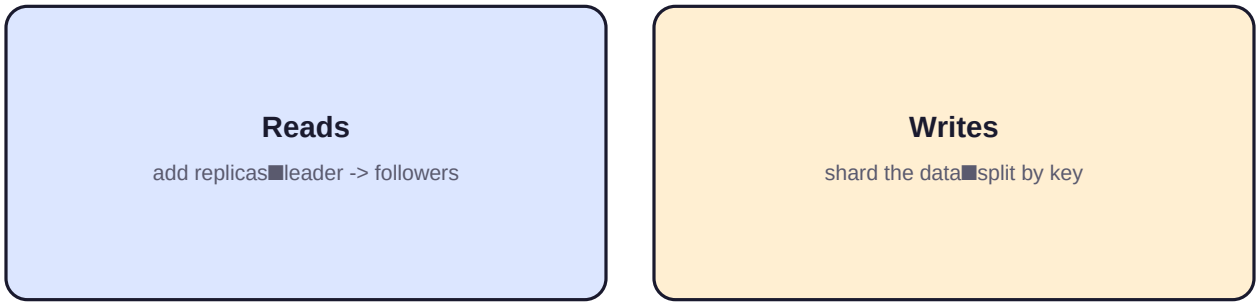
L4 vs L7

L4 routes packets (fast); L7 routes HTTP (path/host-aware, smarter)

STEP 5/6

Scaling the Data Layer

App servers scale out easily. The DATABASE is the hard part.



- Reads scale horizontally with replicas (eventual consistency accepted).
- Writes scale by sharding -- or by choosing a store that shards natively (Cassandra, DynamoDB).
- Caching in front buys 10x headroom: the cheapest scale-out is not hitting the DB at all.

STEP 6/6

Auto-Scaling

Add capacity on signal, remove on calm -- with hysteresis.

scale OUT when: CPU > 70% for 5 min (or queue depth, or p99 latency)

scale IN when: CPU < 30% for 15 min (slower -- avoid flapping)

- Cooldowns prevent thrash: scaling takes minutes (new nodes, warm caches).
- Scale on the RIGHT signal: queue depth and latency beat CPU for bursty workloads.
- Pre-scale for known peaks (Black Friday); keep headroom for unknown ones.

STEP ✓/✓

Key Takeaways

Horizontal vs Vertical Scaling — Remember

- Scale up first: simpler and cheaper until one machine truly cannot cope.
- Scale out with identical stateless servers behind a redundant load balancer.
- Statelessness is mandatory: sessions, files, and jobs live in backing services.
- Reads scale via replicas, writes via sharding; caching buys headroom for both.
- Auto-scale on meaningful signals with cooldowns -- and pre-scale for known peaks.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev